

UNIVERSIDAD POLITÉCNICA DE SAN LUIS POTOSÍ

Ingeniería en Tecnologías de la Información

ROAD TO HALL OF FAME · ACTIVIDAD 10

File Path Traversal

Traversal Sequences Stripped with Superfluous URL-Decode — Nivel Apprentice

Alumno	Manuel Eduardo Hernández Gaytán
Matrícula	181850
Materia	CNO IV – Seguridad Informática
Laboratorio	Actividad 12 · File Path Traversal – Traversal Sequences Stripped with Superfluous URL-Decode
Nivel	Apprentice
Plataforma	PortSwigger Web Security Academy
Docente	Mtro. Servando López Contreras
Fecha	26 de septiembre de 2026

Índice

1. Marco Teórico y Fundamentación Técnica
2. Identificación del Laboratorio
3. Objetivo del Laboratorio
4. Explicación Técnica de la Vulnerabilidad
5. Procedimiento Paso a Paso
6. Capturas de Pantalla Obligatorias
7. Explicación Técnica del Payload
8. Mitigación Recomendada
9. Conclusión

1. Marco Teórico y Fundamentación Técnica

El Path Traversal es una vulnerabilidad de seguridad web que permite a un atacante acceder a archivos arbitrarios del servidor mediante la manipulación de parámetros que construyen rutas del sistema de archivos. Una medida de mitigación habitual consiste en detectar y bloquear secuencias de retroceso de directorio (`../`) presentes en la entrada del usuario. Esta variante del laboratorio explota un escenario en el que dicha validación se aplica correctamente sobre la cadena recibida, pero el propio flujo de la aplicación introduce una decodificación URL adicional e innecesaria ("superflua") después de haber realizado la comprobación.

El parámetro `filename` llega al servidor ya decodificado una primera vez de forma automática por el framework web al procesar la cadena de consulta (`querystring`). Sobre ese valor, la aplicación busca y bloquea secuencias `../` literales. El problema surge porque, más adelante en el flujo, el código que finalmente abre el archivo vuelve a aplicar una decodificación URL sobre el mismo valor. Esta segunda decodificación, sobrante desde el punto de vista de la lógica de la aplicación, ocurre después de que el control de seguridad ya dio su visto bueno, por lo que cualquier secuencia de traversal que solo se materialice tras esa segunda decodificación pasa desapercibida para el filtro.

Estándar	Identificador	Descripción
CWE	CWE-22	Improper Limitation of a Pathname to a Restricted Directory
OWASP Top Ten	A01:2021	Broken Access Control — validación de entradas insuficiente
CVSS v3.x	Base Score: 7.5 (High)	Acceso no autorizado a archivos del sistema sin autenticación
CVSS Vector	AV:N/AC:L/PR:N/UI:N	Explotable remotamente, sin privilegios, sin interacción del usuario

La raíz de la vulnerabilidad reside en aplicar el control de seguridad en un punto del flujo que no es el mismo en el que finalmente se resuelve la ruta. Validar una representación de la entrada (la cadena recibida tras el primer decode) mientras el sistema, más adelante, transforma esa misma entrada en una representación distinta (tras un segundo decode) rompe la garantía que el filtro pretendía ofrecer. El único enfoque robusto es aplicar todas las transformaciones de la entrada (decodificaciones incluidas) antes de validar, y validar exclusivamente sobre la ruta canónica final.

2. Identificación del Laboratorio

Campo	Detalle
Nombre del laboratorio	File Path Traversal — Traversal Sequences Stripped with Superfluous URL-Decode

Campo	Detalle
Nivel	Apprentice
Tipo de explotación	Path Traversal / Double URL-Encoding Bypass
Plataforma	PortSwigger Web Security Academy
Posición en la ruta	Actividad 10 — Road to Hall of Fame
Herramienta principal	Burp Suite Community Edition v2026.8
Protocolo explotado	HTTP/2
Vector de ataque	Parámetro filename del endpoint /image
Técnica de bypass	Doble codificación URL de la barra (%252f) que sobrevive a la validación y se resuelve en la decodificación superflua
URL del laboratorio	https://0ac2002804a5941a835e10c50037003c.web-security-academy.net/

3. Objetivo del Laboratorio

Objetivo general:

Demostrar de forma práctica y documentada la explotación de una variante de Path Traversal en la que la aplicación bloquea correctamente las secuencias `../` literales, pero aplica una decodificación URL superflua después de dicha validación. Mediante la construcción de un payload con doble codificación URL de la barra (`..%252f..%252f..%252fetc/passwd`), que no contiene ninguna secuencia `../` literal en el momento de la comprobación, se recuperará el contenido del archivo `/etc/passwd` del servidor objetivo.

Objetivos específicos

- Identificar la petición HTTP vulnerable con el parámetro filename (GET `/image?filename=65.jpg`) en el historial de Burp Suite.
- Interceptar la petición y enviarla al módulo Repeater para su modificación controlada.
- Construir el payload `..%252f..%252f..%252fetc/passwd`, cuya doble codificación URL evita ser detectado como secuencia de traversal en la validación.
- Verificar que el servidor responde con HTTP/2 200 OK y devuelve el contenido de `/etc/passwd`.
- Obtener la confirmación 'Lab Solved' de la plataforma PortSwigger.

4. Explicación Técnica de la Vulnerabilidad

La aplicación del laboratorio es una tienda en línea que sirve imágenes de producto a través del siguiente endpoint:

```
GET /image?filename=65.jpg HTTP/2
Host: <lab-id>.web-security-academy.net
```

Cuando el servidor recibe la petición, el framework decodifica automáticamente la cadena de consulta (primer decode) antes de entregar el valor de filename a la lógica de la aplicación. Sobre ese valor ya decodificado, la aplicación busca secuencias `../` literales y las bloquea. Hasta aquí, el control de seguridad parece robusto: si el atacante envía `../../etc/passwd`, la validación lo detecta sin problema.

Mecanismo del bypass: la decodificación superflua. El problema aparece en un punto posterior del flujo: antes de abrir el archivo, la aplicación vuelve a aplicar una decodificación URL sobre el mismo valor de filename, de forma innecesaria (superflua), ya que dicho valor ya había sido decodificado una vez por el framework. Esta segunda decodificación ocurre después de que el filtro de traversal ya emitió su veredicto, por lo que cualquier secuencia que solo se convierta en `../` tras ese segundo decode elude por completo la validación.

Valor del parámetro	Tras el primer decode (validación)	Tras el segundo decode (superfluo)
65.jpg	65.jpg — sin traversal, pasa	65.jpg — sirve la imagen
../../etc/passwd	../../etc/passwd — detectado y bloqueado	No aplica — la petición ya fue bloqueada
..%252f..%252f..%252fetc/passwd	..%2f..%2f..%2fetc/passwd — sin '../' literal, pasa	../../etc/passwd — accede a /etc/passwd

5. Procedimiento Paso a Paso

Paso 1: Acceso al laboratorio y configuración de Burp Suite

Se accedió al laboratorio "File path traversal, traversal sequences stripped with superfluous URL-decode" en PortSwigger Web Security Academy, disponible en <https://0ac2002804a5941a835e10c50037003c.web-security-academy.net/>. Se abrió Burp Suite Community Edition con la configuración predeterminada, con el proxy local activo en 127.0.0.1:8080.

Paso 2: Carga de la página en el navegador de Burp Suite

Desde la pestaña Proxy se abrió el navegador integrado de Burp Suite ('Open Browser'). Se navegó a la URL del laboratorio y se esperó a que la tienda en línea ('We Like To Shop') cargara completamente, quedando todo el tráfico disponible en el historial HTTP de Burp Suite.

Paso 3: Identificación del vector de ataque

En la pestaña Proxy > HTTP history se identificó la petición GET `/image?filename=65.jpg`, correspondiente a la carga de una imagen de producto. Esta petición contiene el parámetro filename, que es el vector de ataque.

Paso 4: Envío de la petición original al Repeater

La petición se envió al módulo Repeater de Burp Suite (Ctrl+R), donde se visualizó la petición original antes de cualquier modificación:

```
GET /image?filename=65.jpg HTTP/2
Host: 0ac2002804a5941a835e10c50037003c.web-security-academy.net
```

Paso 5: Construcción y envío del payload de bypass

En el Repeater se modificó el valor del parámetro filename, sustituyendo el nombre de la imagen por un payload con doble codificación URL de la barra, de modo que ninguna secuencia ../ literal esté presente en el momento de la validación. Se presionó el botón 'Send':

```
# Petición modificada con payload de bypass:
GET /image?filename=..%252f..%252f..%252fetc/passwd HTTP/2
```

Paso 6: Análisis de la respuesta

El panel de respuesta del Repeater mostró el código HTTP/2 200 OK y el cuerpo de la respuesta contenía el archivo completo /etc/passwd del sistema operativo del servidor, incluyendo la lista de usuarios del sistema (root, daemon, peter, carlos, entre otros).

Paso 7: Confirmación — Lab Solved

Se actualizó la página en el navegador integrado de Burp Suite y la plataforma PortSwigger mostró el banner de confirmación 'Congratulations, you solved the lab!'. Se verificó adicionalmente el mismo resultado en un navegador externo, confirmando de forma consistente el éxito del bypass.

6. Capturas de Pantalla Obligatorias

Se presentan las siete capturas de pantalla que documentan visualmente el proceso completo de explotación, desde el estado inicial del laboratorio hasta la doble confirmación de su resolución.

Captura 1 — Estado inicial del laboratorio

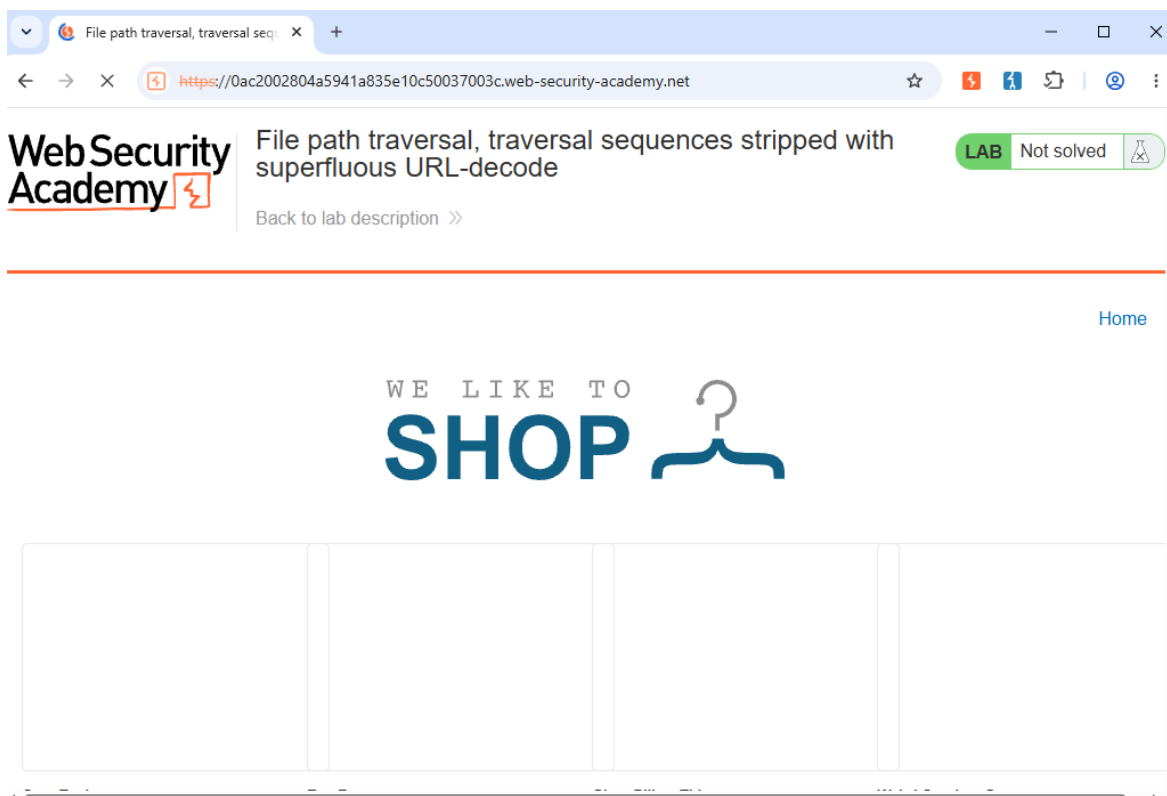


Figura 1. Estado inicial del laboratorio 'File path traversal, traversal sequences stripped with superfluous URL-decode' en PortSwigger Web Security Academy, marcado como 'Not solved'.

Captura 2 — Petición identificada en el historial HTTP

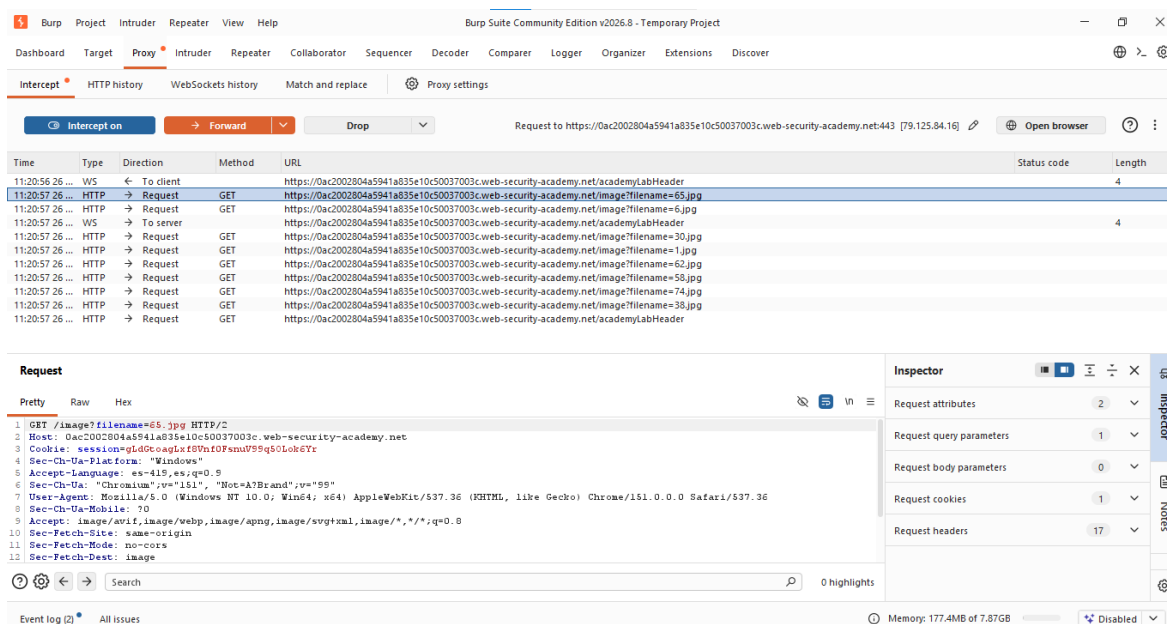


Figura 2. Historial HTTP de Burp Suite mostrando la petición GET /image?filename=65.jpg identificada como vector de ataque.

Captura 3 — Petición original en el Repeater

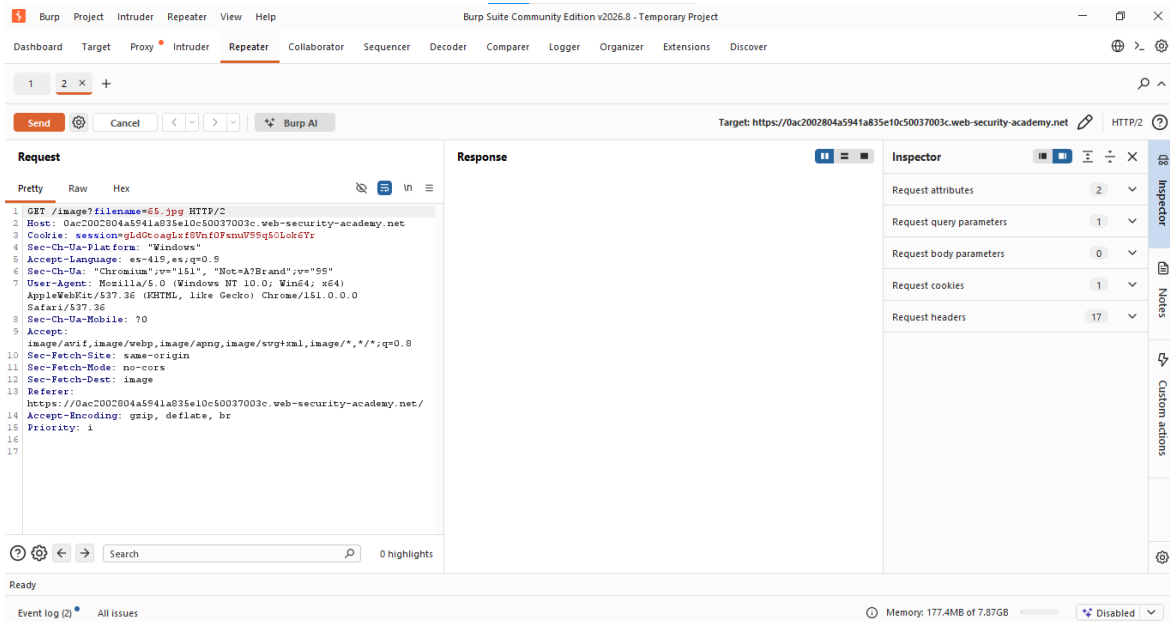


Figura 3. Petición original en el módulo Repeater de Burp Suite, lista para ser modificada.

Captura 4 — Payload de bypass enviado al servidor

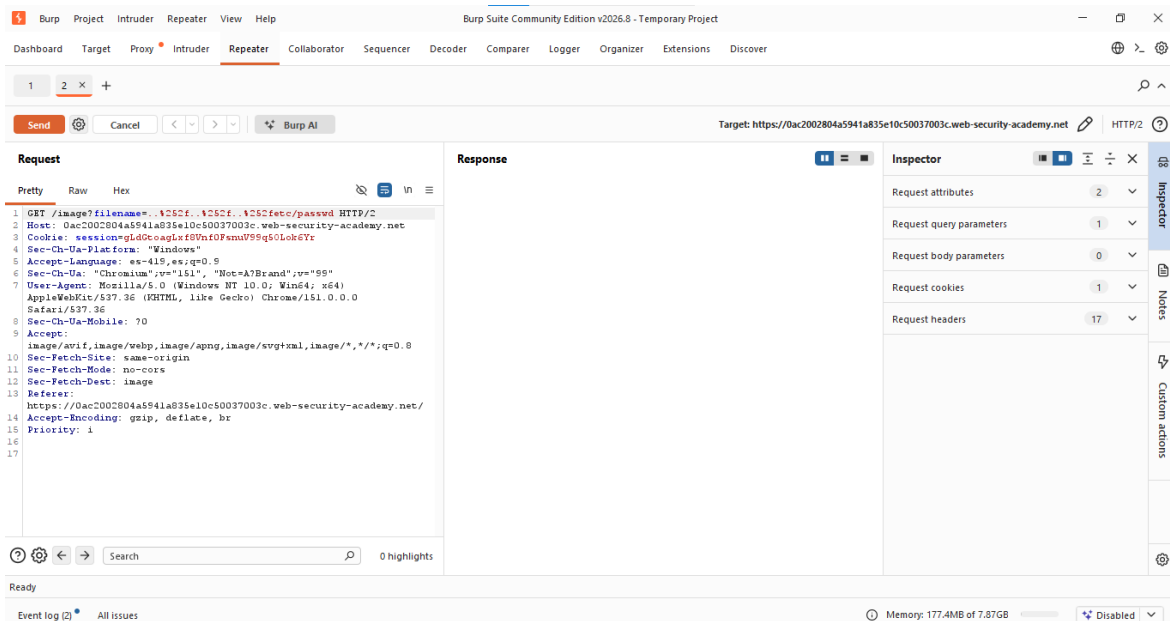


Figura 4. Petición modificada con el payload de doble codificación URL: `GET /image?filename=../../%252f../%252f../%252fetc/passwd HTTP/2`.

Captura 5 — Respuesta del servidor con el contenido de `/etc/passwd`

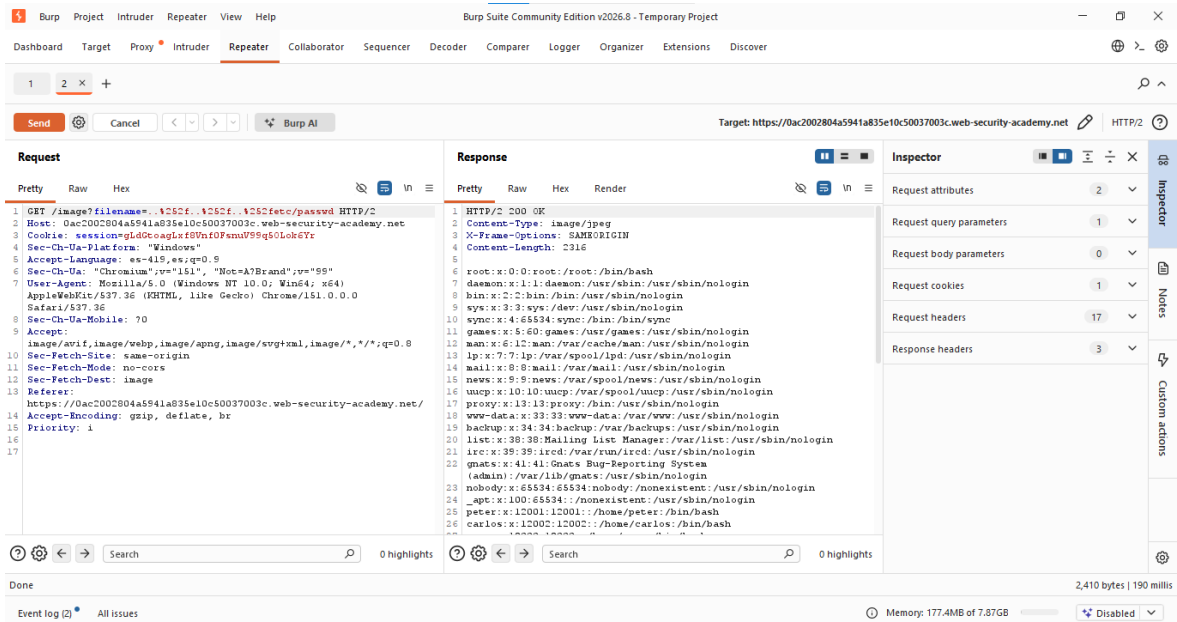


Figura 5. Respuesta HTTP/2 200 OK con el contenido completo del archivo /etc/passwd, incluyendo los registros de usuarios del sistema.

Captura 6 — Confirmación 'Lab Solved' en el navegador de Burp Suite

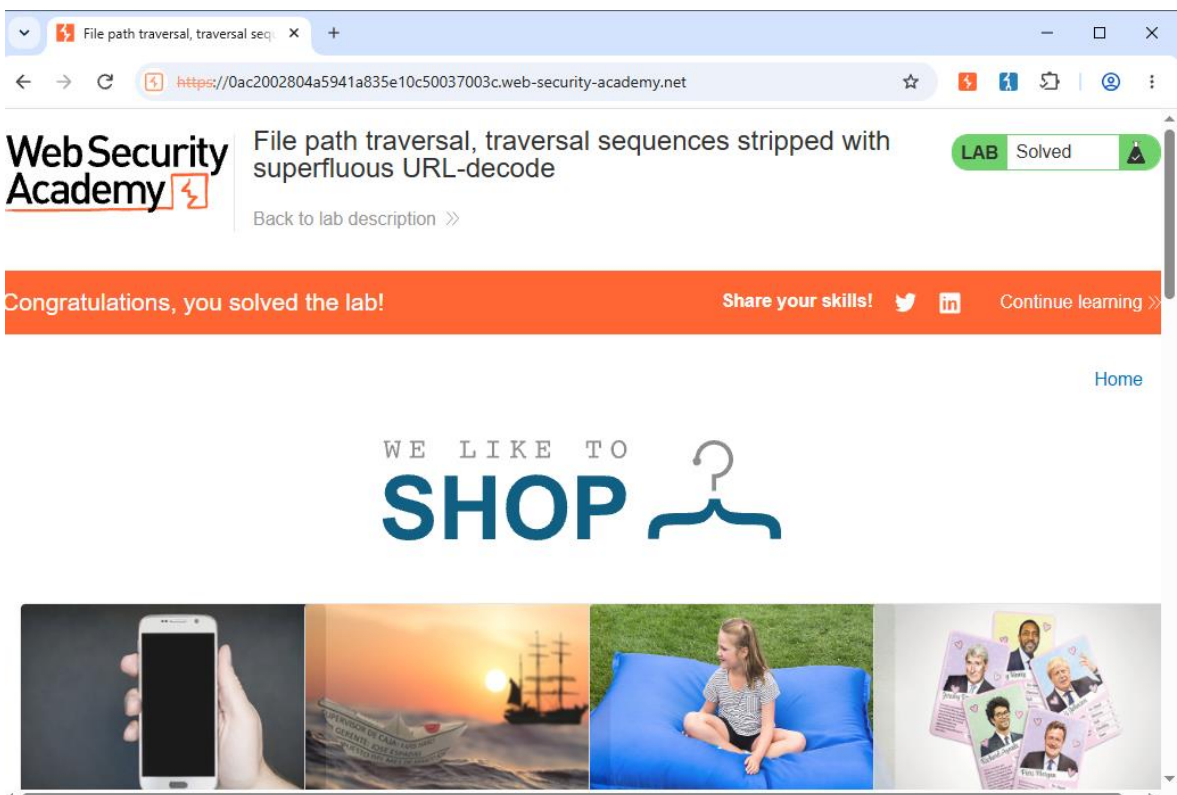


Figura 6. Banner de confirmación 'Congratulations, you solved the lab!' en el navegador integrado de Burp Suite.

Captura 7 — Verificación final en navegador externo

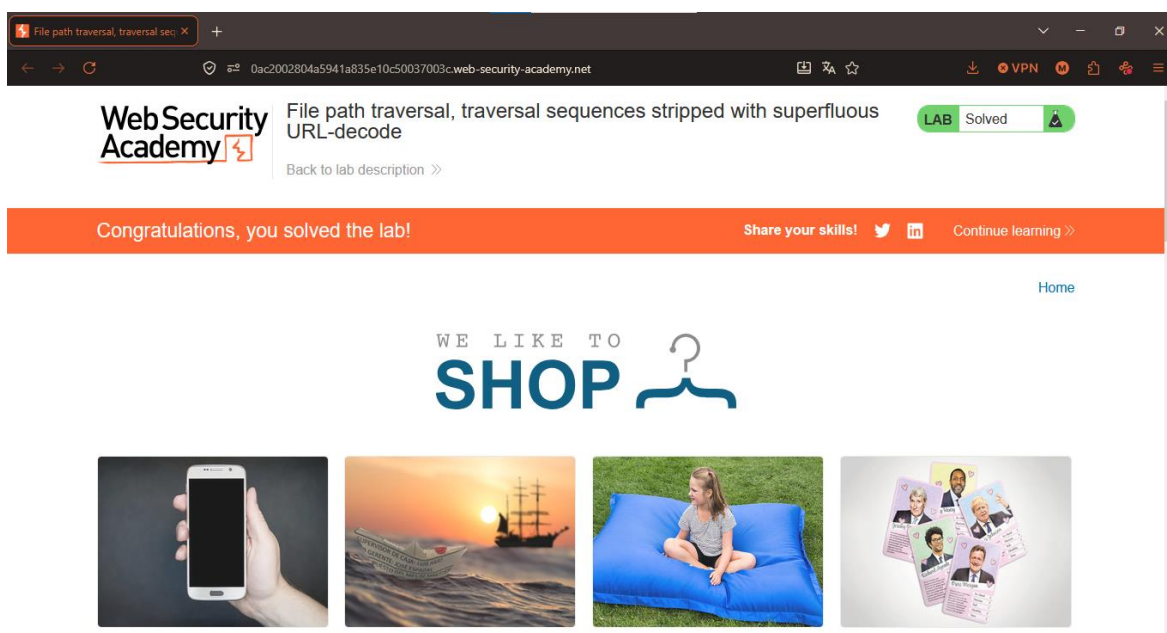


Figura 7. Verificación final del banner 'Congratulations, you solved the lab!' en un navegador externo, confirmando la resolución del laboratorio.

7. Explicación Técnica del Payload

```
..%252f..%252f..%252fetc/passwd
```

El payload se construye codificando dos veces la barra (/) de cada secuencia de retroceso de directorio, dejando los dos puntos (..) sin codificar. El carácter % se codifica como %25, de modo que %2f (la barra codificada una vez) se convierte en %252f al codificarla una segunda vez:

Etapas	Valor de un bloque	Contiene '../' literal
Payload enviado	..%252f	No
Tras el primer decode (el que valida el filtro)	..%2f	No
Tras el segundo decode (superfluo, ya sin validar)	../	Sí — pero ya es demasiado tarde

¿Por qué no es necesario codificar los puntos? El filtro de este laboratorio busca específicamente la secuencia literal ../ completa. Basta con que uno solo de sus tres caracteres (la barra) no esté presente en forma literal en el momento de la validación para que la secuencia no sea reconocida como traversal. Codificar doblemente solo la barra es suficiente y mantiene el payload más simple; codificar también los puntos habría funcionado igual, pero no era necesario.

Con tres bloques ..%252f repetidos antes de etc/passwd, tras la decodificación superflua la ruta se reconstruye como ../../etc/passwd, suficiente para retroceder desde el directorio de imágenes hasta la raíz del sistema de archivos y acceder al archivo objetivo.

8. Mitigación Recomendada

8.1 Canonicalización y verificación de la ruta resultante (prioridad crítica)

La medida más robusta consiste en aplicar todas las transformaciones de la entrada — incluida cualquier decodificación URL— antes de realizar cualquier validación, y validar exclusivamente sobre la ruta canónica final. De esta forma no existe una ventana entre 'lo que se valida' y 'lo que finalmente se abre':

```
import os
from urllib.parse import unquote

BASE_DIR = os.path.realpath('/var/www/images')

def get_image(raw_filename: str):
    # Decodificar por completo ANTES de validar (nunca después)
    filename = unquote(unquote(raw_filename))
    full_path = os.path.join(BASE_DIR, os.path.basename(filename))
    canonical = os.path.realpath(full_path)
    if not canonical.startswith(BASE_DIR + os.sep):
        raise PermissionError('Acceso denegado')
    return open(canonical, 'rb').read()
```

8.2 Validación por allowlisting del valor del parámetro (prioridad crítica)

Definir exactamente qué valores son aceptables para el parámetro filename — un nombre de archivo simple, sin barras ni secuencias codificadas — elimina la superficie de ataque, independientemente de cuántas veces se decodifique la entrada en el flujo de la aplicación:

```
import re

ALLOWED = re.compile(r'^[a-zA-Z0-9_-]+\.(jpg|jpeg|png|gif|webp)$')

def validate(filename: str) -> str:
    if not ALLOWED.match(filename):
        raise ValueError('Nombre de archivo no permitido')
    return filename
```

8.3 Por qué una decodificación superflua rompe la validación

Un control de seguridad solo es confiable si se aplica sobre la representación final y definitiva de la entrada, es decir, después de que todas las transformaciones (decodificaciones, normalizaciones, etc.) hayan ocurrido. Si el flujo de la aplicación decodifica la entrada en más de un punto, y la validación solo cubre uno de ellos, cualquier transformación adicional posterior es una oportunidad para reintroducir el patrón que se pretendía bloquear.

Técnica de bypass	Ejemplo	¿Lo detiene la validación de un solo decode?	¿Lo detiene decodificar antes de validar?
Traversal literal	../../etc/passwd	Sí	Sí
Codificación URL simple	..%2f..%2f..%2fetc/passwd	Depende del orden decode/validación	Sí
Doble codificación URL	..%252f..%252f..%252fetc/passwd	No	Sí
Codificación mixta de puntos y barra	%2e%2e%252fetc/passwd	No	Sí

8.4 Principio de mínimo privilegio y diseño seguro del endpoint

- Ejecutar el servidor web con un usuario sin privilegios que solo tenga permisos de lectura sobre el directorio de archivos estáticos.
- Aplicar controles a nivel del sistema operativo (chroot, namespaces, contenedores) para confinar el proceso del servidor web.
- Evitar exponer rutas de archivo en parámetros de URL; mapear internamente un identificador abstracto (ID numérico o UUID) al archivo real.
- Usar funciones del framework para servir archivos estáticos (send_from_directory en Flask, StaticFiles en FastAPI, X-Accel-Redirect en Nginx), que incluyen validación de ruta nativa y evitan decodificaciones manuales duplicadas.

9. Conclusión

La realización de este laboratorio permitió comprender una variante particular de bypass de Path Traversal en la que el propio filtro de seguridad funcionaba correctamente sobre la entrada que recibía, pero el flujo de la aplicación introducía una decodificación URL adicional después de dicha validación. Al enviar un payload con la barra doblemente codificada (..%252f..%252f..%252fetc/passwd), la cadena evaluada por el filtro nunca contuvo una secuencia ../ literal, y solo tras la decodificación superflua posterior se materializó el traversal que permitió leer /etc/passwd.

El caso ilustra un principio de seguridad fundamental: un control de validación es tan confiable como el punto exacto del flujo en el que se aplica. Si la entrada sufre transformaciones adicionales después de haber sido validada, esas transformaciones son una superficie de ataque en sí mismas, independientemente de cuán robusta parezca la validación original. Encadenar decodificaciones sin garantizar que la validación ocurra después de la última de ellas convierte un filtro aparentemente correcto en una defensa ineficaz.

El impacto del exploit es el mismo que en otras variantes de Path Traversal: lectura no autorizada del archivo /etc/passwd con la lista completa de usuarios del sistema operativo del servidor, información que

puede aprovecharse para escalar el ataque mediante enumeración de cuentas, ataques de fuerza bruta y, en escenarios más complejos, lectura de archivos de configuración con credenciales de acceso.

La única mitigación verdaderamente robusta frente a esta familia de vulnerabilidades es garantizar que todas las decodificaciones de la entrada ocurran antes de validar, y verificar siempre el confinamiento al directorio base sobre la ruta canónica final, en lugar de sobre una representación intermedia de la cadena recibida.